

Scalable Fine-tuning from Multiple Data Sources:

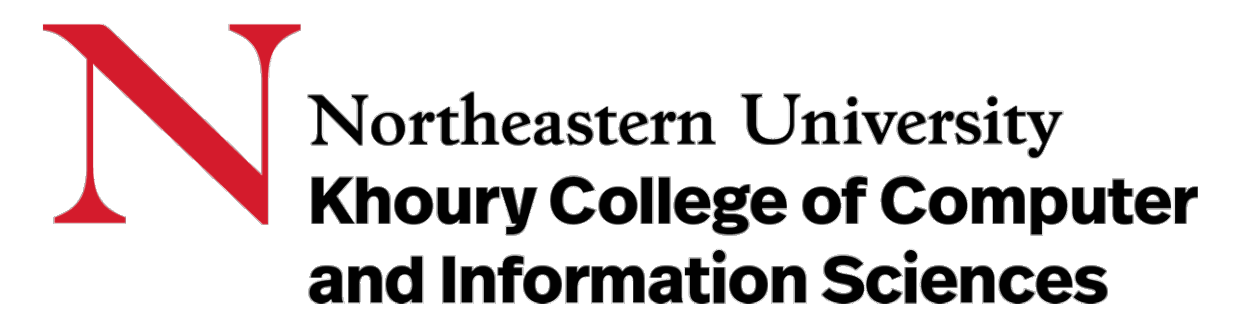
A First-Order Approximation Approach

Dongyue Li^{†*}, Ziniu Zhang^{†*}, Lu Wang[‡], Hongyang R. Zhang[†]

[†]Northeastern University, Boston

[‡]University of Michigan, Ann Arbor

[†]{li.dongyu, zhang.zini, ho.zhang}@northeastern.edu [‡]wangluxy@umich.edu



BACKGROUND

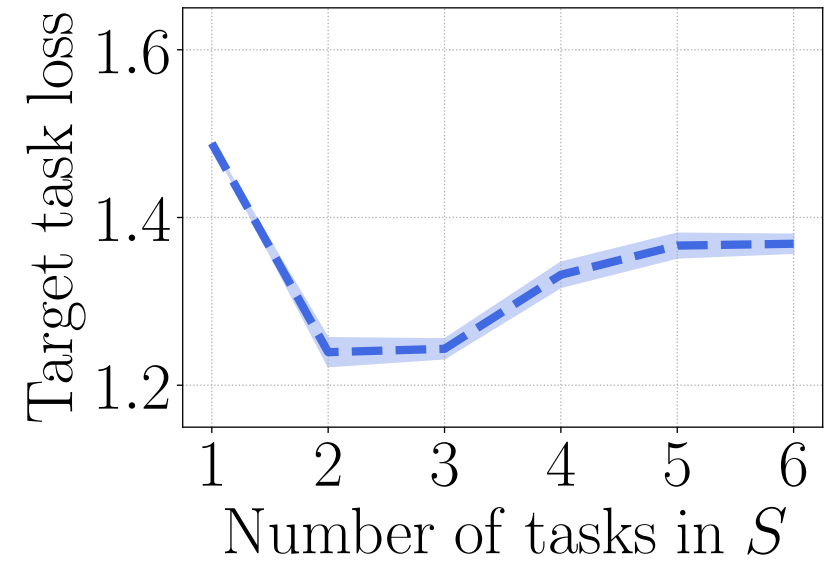
Fine-tuning language models involves many data sources in many applications:

- Instruction tuning language models on a massive set of multiple tasks, such as the FLAN collection (Longpre et al., '23).
- Fine-tuning with LLM-generated contents, e.g., chain-of-thoughts (Ho et al., '23).
- Training multilingual models, e.g., neural machine translation (Neubig et al. '18).

Given n tasks, find a subset $S \subseteq \{1, 2, \dots, n\}$ that minimizes the fine-tuning model loss on a target task, denoted as $f(S)$.

Examples: Fine-tuning a Llama-1B model on the Alpaca dataset with 38 tasks.

- 47% pairwise combination of tasks leads a negative transfer on the target task, i.e., increasing $f(S)$
- $f(S)$ is not monotone: gradually adding “beneficial” tasks to S can even increase $f(S)$.
- Gradients or representation similarity scores weakly correlate with $f(S)$. The correlation is less than 0.2.



Canonical subset selection methods:

	Forward selection (Hastie et al. '08)	Random ensemble (Ilyas et al. '22)
Steps	1. Enumerate the tasks that have not yet been added to the subset. 2. Compute the value of f if each task were to be included. 3. Add the task corresponding to the minimum f value to the subset	1. Sample m random subsets from $\{1, \dots, n\}$. 2. Fit one model on each task subset. 3. Compute a score for each task j as the average $f(S)$ among subsets that contain j.
Subsets	$\{1\}, \{2\}, \dots, \{1, 2\}, \{1, 3\}, \dots, \{1, 2\dots n\}$	Randomly sampled subsets of a fixed size from $\{1, 2, \dots, n\}$

Problem statement: Applying the subset selection methods requires repeatedly fine-tuning LMs on numerous subsets, which is computationally expensive.

Given a list of task subsets, can we quickly estimate $f(S)$, without repeated fine-tuning language models on each subset?

OUR APPROACH

Our key idea is to replace fine-tuning with a *regression-based first-order estimation of the fine-tuning model loss* on a particular subset of tasks.

1. Obtain a meta-initialization by multitask training on all tasks, denoted as θ^*
2. Estimate the LM fine-tuning loss for every subset S_i , by solving logistic regression using gradients on θ^* as features.

First-order approximation of fine-tuned models: We examine first-order Taylor's expansion of a fine-tuned model $h_X(s, y)$ from the initialization θ^* :

$$h_X(s, y) \approx h_{\theta^*}(s, y) + [\nabla_X h_{\theta^*}(s, y)]^T (X - \theta^*) + \epsilon_s$$

We find the error ϵ_s remains negligible for X around θ^* . When the relative distance from X to θ^* is within 10%, the relative error is less than 1%.

Dist.	Pythia-70M	BERT-Base	RoBERTa-Base	GPT-2
10%	$7_{\pm 1.4} \times 10^{-3}$	$5_{\pm 1.4} \times 10^{-3}$	$5_{\pm 0.5} \times 10^{-3}$	$5_{\pm 0.4} \times 10^{-3}$
Dist.	FLAN-T5-Base	BloomZ-560M	TinyLlama-1.1B	GPT-Neo-1.3B
10%	$5_{\pm 1.2} \times 10^{-3}$	$5_{\pm 2.2} \times 10^{-3}$	$5_{\pm 0.8} \times 10^{-3}$	$5_{\pm 0.8} \times 10^{-3}$
Dist.	OPT-1.3B	Gemma-2-2B	Mistral-7B	Llama-3-8B
10%	$5_{\pm 0.1} \times 10^{-3}$	$4_{\pm 0.5} \times 10^{-3}$	$4_{\pm 0.1} \times 10^{-3}$	$5_{\pm 0.4} \times 10^{-4}$

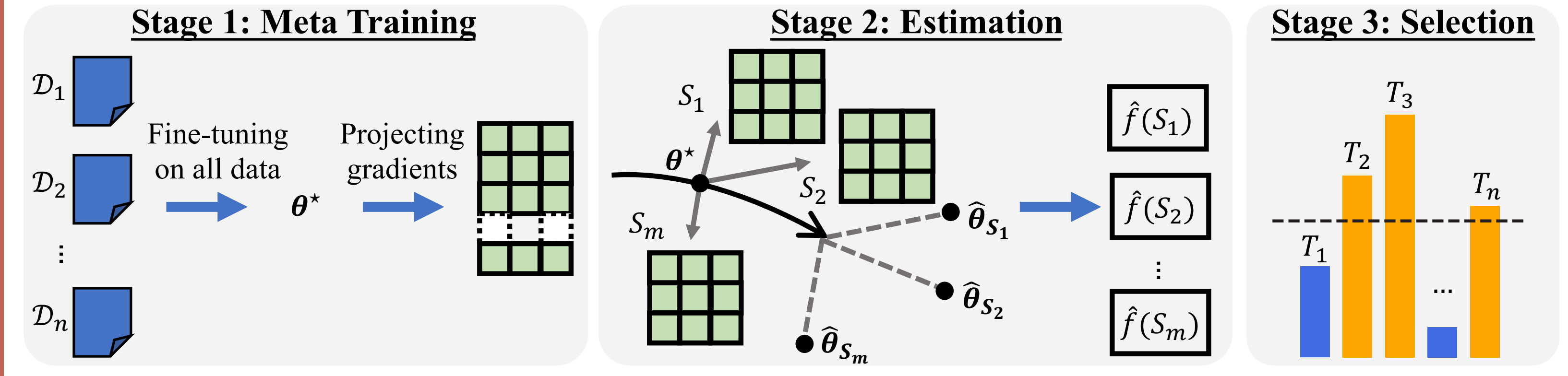
Fast estimation for each subset using gradients: Apply the first-order approximation to the log-loss of the model:

$$\tilde{\ell}(X) = \log(1 + \exp(b - y \cdot g^T(X - \theta^*)))$$

The estimation becomes solving logistic regression using the gradient g_i as the feature vector and the task label y as the response label.

We project the gradients to a low dimension using the Johnson-Lindenstrauss lemma. Solving each logistic regression performs on CPUs and takes a few seconds.

SUMMARY OF OUR APPROACH



Our algorithm: Fast Estimation of LM Fine-Tuning Losses Using Gradients (GRADEx)

- GRADEx reduces the $m = \Omega(n)$ model fine-tuning to 1.
- GradEx trades-off model fine-tuning with $O(n)$ gradient computation and solving logistic regression m times.

Methods	Measure	Runtime
Forward Selection (FS)	$f(S)$	$O(n^3)$
Random Ensemble (RE)	$f(S)$	$O(n \log n)$
DSIR (Xie et al. '23)	n -gram features	$O(n)$
DEFT (Iverson et al. '23)	Feature similarity	$O(n)$
LESS (Xia et al. '24)	Gradient similarity	$O(n)$
GRADEx	$f(S)$	$O(n)$

EXPERIMENTS

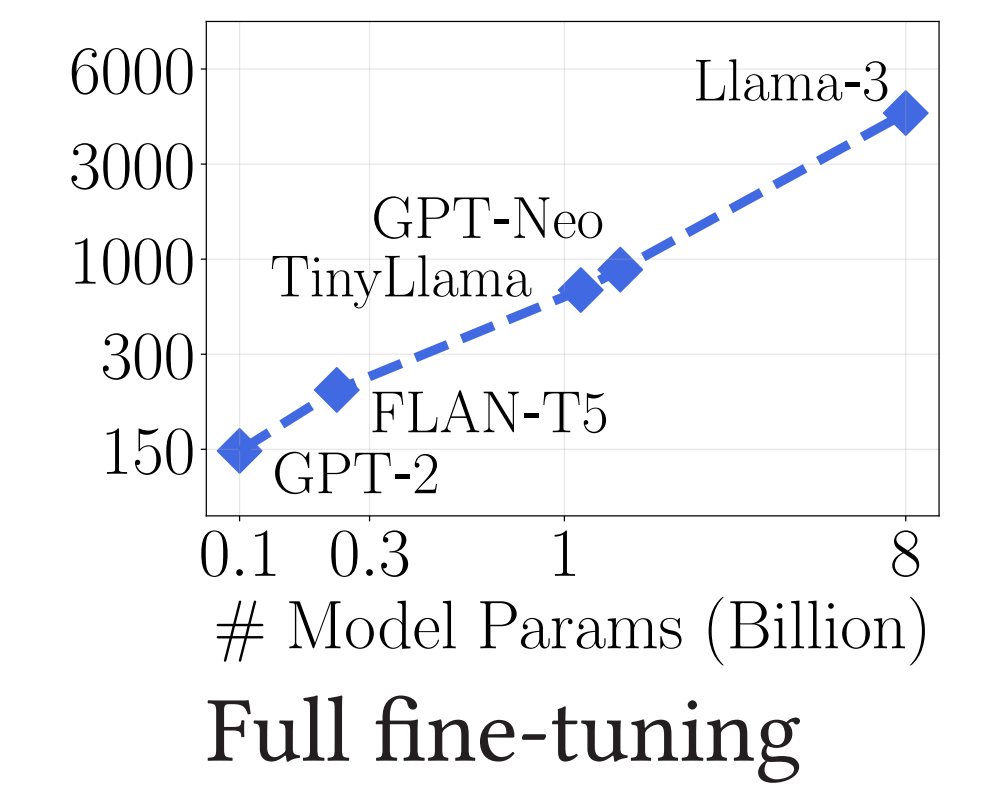
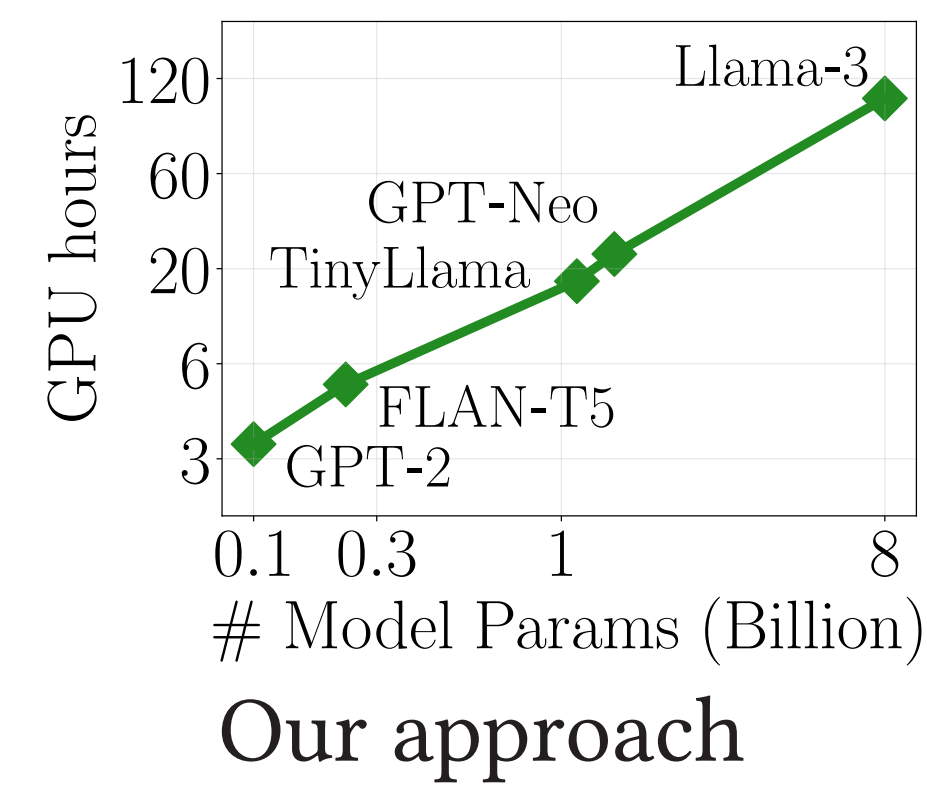
Approximating model fine-tuning losses: We report the relative distance between the estimated loss $\hat{f}(S)$ and $f(S)$.

- GRADEx approximates the fine-tuning losses within **1%** relative error.
- GRADEx accelerates forward selection by **30×** in FLOPs and **25×** in GPU hours.
- GRADEx accelerates random ensemble by **44×** in FLOPs and **46×** in GPU hours.

Alpaca	TinyLlama-1.1B	GPT-Neo-1.3B	Llama-3-8B	Speedup
GRADEx-FS	3.9×10^{-4}	3.5×10^{-4}	2.7×10^{-4}	17.6×
GRADEx-RE	4.2×10^{-4}	3.8×10^{-4}	2.9×10^{-4}	43.3×

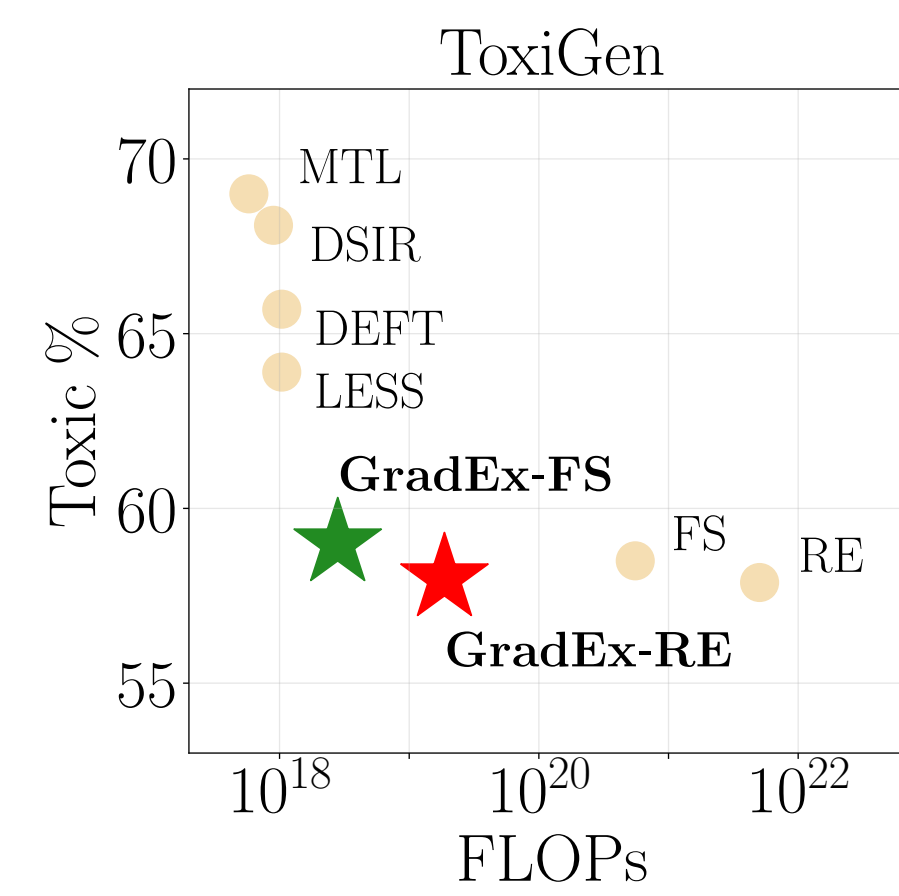
StrategyQA	TinyLlama-1.1B	GPT-Neo-1.3B	Llama-3-8B	Speedup
GRADEx-FS	3.2×10^{-4}	3.0×10^{-4}	2.4×10^{-4}	30.5×
GRADEx-RE	3.8×10^{-4}	3.4×10^{-4}	2.8×10^{-4}	44.8×

The speed-up is consistent across different models, as it is mainly from the reduction of fine-tuned models.

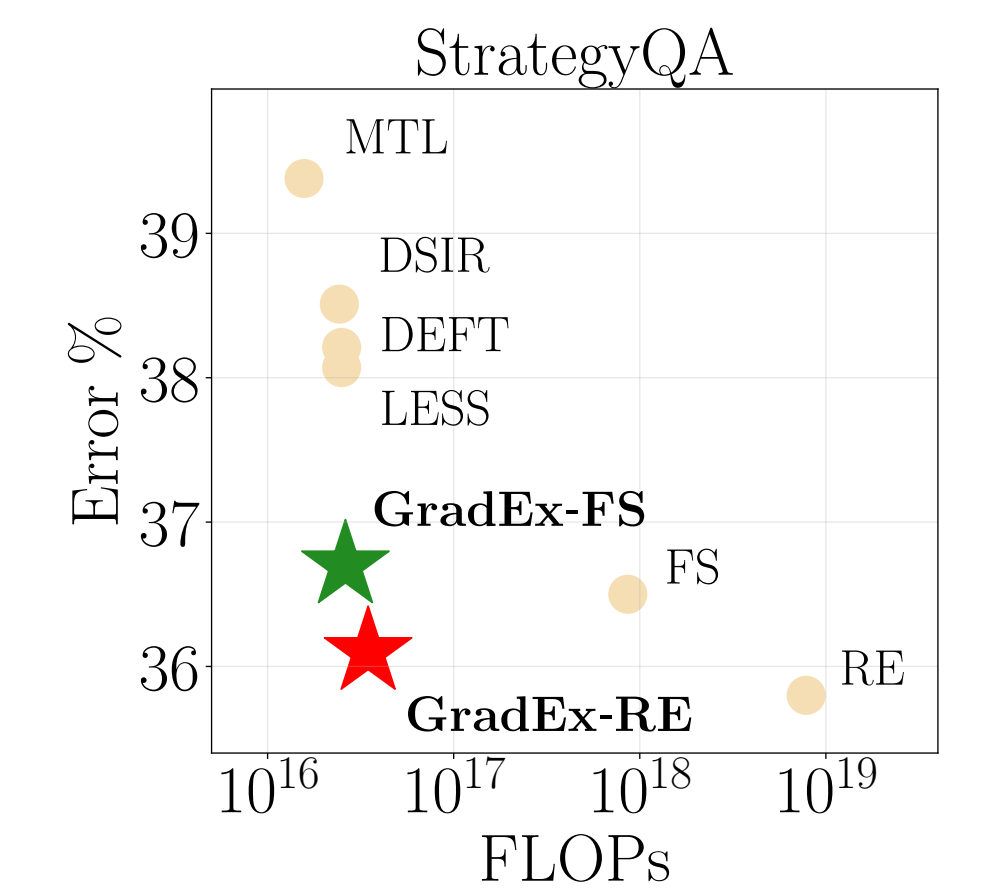


Downstream performance evaluation:

- Our approach delivers a comparable test accuracy to classic subset selection methods, incurring less than **0.5%** computation cost.
- Our approach outperforms existing selection methods (DSIR, DEFT, and LESS) by up to **3.8%**, with comparable computational costs.

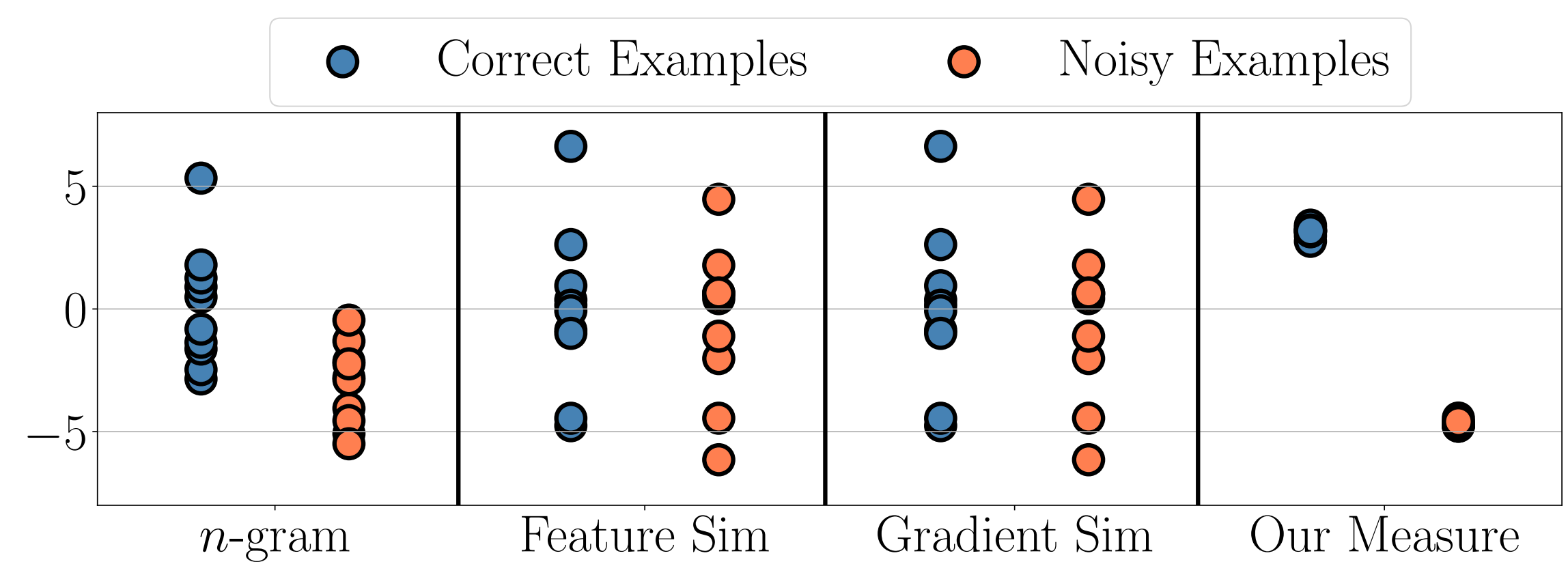


Instruction tuning



Chain-of-thought fine-tuning

Example: The measure estimated by GRADEx leads to a clearer separation of correct vs noisy examples, on a synthetic addition task with label noise.



Code: <https://github.com/VirtuosoResearch/Scalable-finetuning>

Paper: <https://arxiv.org/pdf/2409.19458>